

bomblab 报告

姓名：陈若闻婧

学号：2024201580

总分	phase_1	phase_2	phase_3	phase_4	phase_5	phase_6	secret_phase
7	1	1	1	1	1	1	1

2024201580	0	0	0	0	0	0	0	0	0	0	0
------------	---	---	---	---	---	---	---	---	---	---	---

解题报告

phase_1

A harmony of Light awaits you in a lost world of musical Conflict.

讲解题目思路

输入作为第一个参数，通过`lea 0x1d40(%rip),%rsi`，把`0x1d40(%rip)`这个地址放入寄存器`rsi`中作为`strings_not_equal`的第二个参数，返回值在`eax`中，如果这两个字符串相等返回0，不相等返回1。后面的`jne`跳转的意思是如果`eax`不等于0，就爆炸，所以说明不爆炸的条件是两个字符串相同，所以正确的输入就是`0x1d40(%rip)`这个地址中存储的字符串，可以通过看寄存器`rsi`所指地址存储的字符串来找到答案。

以下是伪代码：

```
void phase_1(char *input) {
    // 硬编码在内存中的标准答案字符串
    // 地址通过 lea 0x1d40(%rip), %rsi 加载
    char *secret_string = "A harmony of Light awaits you in a lost world of musical Conflict.";

    // 调用 strings_not_equal 比较输入 (%rdi) 和 答案 (%rsi)
    // 如果返回 1 (不相等)，则爆炸
    if (strings_not_equal(input, secret_string)) {
        explode_bomb();
    }
    // 否则过关
    return;
}
```

phase_2

739687 561581 869917 662790

讲解题目思路

Phase 2 的核心逻辑是：程序首先读取用户输入的 **4 个整数**（如果输入数量不对也会爆炸）。随后，程序内部利用两个预置的全局数组（矩阵 matA 和 matB）进行矩阵乘法，将计算结果存储在栈上的临时空间中。最后，将输入的 4 个数与程序计算出的 4 个结果比对，一样才能通过，否则爆炸。

第一阶段：检查输入

```
146e: lea 0x4(%rsp), %rcx    ; 第2个参数
1473: lea 0xc(%rsp), %r9      ; 第4个参数
...
1484: call sscanf             ; 调用 sscanf
1489: cmp $0x4, %eax          ; 检查 sscanf 的返回值
148c: jne 14a2                ; 如果返回值不等于 4，跳转至爆炸路径
```

先传参，然后调用sscanf，如果返回值（也就是输入个数不等于4，爆炸，所以正确的输入有四个数字）

第二阶段：矩阵乘法

```
148e: 48 8d 3d ab 4c 00 00    lea    0x4cab(%rip),%rdi #6140 <matA.3>; 加载
数组A的地址
1495: 48 8d 5c 24 10          lea    0x10(%rsp),%rbx#从%rsp+16的位置开始存放矩
阵计算结果，所以以下用%rbx存储结果的地址
149a: 41 bb 00 00 00 00       mov    $0x0,%r11d    #最外层的计数器
14a0: eb 19                  jmp    14bb <phase_2+0x66>
14bb: 48 8d 35 5e 4c 00 00    lea    0x4c5e(%rip),%rsi #6120 <matB.2>;加载数
组B的地址
```

在第一次循环时并未用到这段代码：

```
14a9: 41 83 c3 01            add    $0x1,%r11d    #最外层累加器，控制A的行，第
一层循环开始
14ad: 48 83 c7 0c            add    $0xc,%rdi     #矩阵A每行4个int
14b1: 48 83 c3 08            add    $0x8,%rbx     #每一次外层循环，得到两个int
14b5: 41 83 fb 02            cmp    $0x2,%r11d    #%r11d要到2才结束循环，一共
循环2次
14b9: 74 47                  je     1502 <phase_2+0xad>
```

第一次循环从这里进入（对汇编的详细分析在右侧注释）：

```
14c2: 49 89 d9              mov    %rbx,%r9      #放结果的地址
14c5: 41 b8 00 00 00 00     mov    $0x0,%r8d     #第二层循环累加器，控制B的列
14cb: 4d 89 ca              mov    %r9,%r10      #第二层循环开始
14ce: b8 00 00 00 00       mov    $0x0,%eax     #最内层循环
14d3: b9 00 00 00 00       mov    $0x0,%ecx     #eax,ecx置0
14d8: 8b 14 87              mov    (%rdi,%rax,4),%edx #取矩阵A的数
14db: 0f af 14 c6          imul   (%rsi,%rax,8),%edx #取矩阵B的数，矩阵B每
行只有2个int
14df: 01 d1                add    %edx,%ecx     #ecx是结果累加器
14e1: 48 83 c0 01          add    $0x1,%rax     #rax加一，表示开始对第二个数
计算
```

14e5:	48 83 f8 03	cmp	\$0x3,%rax	#最内层循环三次
14e9:	75 ed	jne	14d8 <phase_2+0x83>	#循环, 直到rax=3
14eb:	41 89 0a	mov	%ecx, (%r10)	#把结果%ecx放在内存中
14ee:	41 83 c0 01	add	\$0x1,%r8d	#第二层循环累加器
14f2:	49 83 c1 04	add	\$0x4,%r9	#因为存的是int, 所以每次放结
果的地址+4				
14f6:	48 83 c6 04	add	\$0x4,%rsi	#矩阵B的下一列首地址
14fa:	41 83 f8 02	cmp	\$0x2,%r8d	#第二层循环两次
14fe:	75 cb	jne	14cb <phase_2+0x76>	
1500:	eb a7	jmp	14a9 <phase_2+0x54>	
1502:	bb 00 00 00 00	mov	\$0x0,%ebx	#结束循环
1507:	48 8d 6c 24 10	lea	0x10(%rsp),%rbp	
150c:	eb 0a	jmp	1518 <phase_2+0xc3>	
150e:	48 83 c3 04	add	\$0x4,%rbx	#循环比较四个输入
1512:	48 83 fb 10	cmp	\$0x10,%rbx	
1516:	74 10	je	1528 <phase_2+0xd3>	#比较完4个数就跳
到函数结尾处				
1518:	8b 44 1d 00	mov	0x0(%rbp,%rbx,1),%eax	
151c:	39 04 1c	cmp	%eax, (%rsp,%rbx,1)	#把输入和矩阵计算结
果对比				
151f:	74 ed	je	150e <phase_2+0xb9>	
1521:	e8 92 09 00 00	call	1eb8 <explode_bomb>	#不等于就爆炸

综合上面对汇编的详细分析，总结：

- **外层循环 (i)：** 控制矩阵A的行。
 - 计数器：%r11d (初始化为 0)
 - 结束条件：14b5: cmp \$0x2, %r11d (循环 2 次)
 - **推论：** 结果矩阵有 **2 行**。
- **内层循环 (j)：** 控制矩阵B的列。
 - 计数器：%r8d (初始化为 0)
 - 结束条件：14fa: cmp \$0x2, %r8d (循环 2 次)
 - **推论：** 结果矩阵有 **2 列**。
- **最内层计算循环 (k)：** A的列和B的行。
 - 计数器：%eax (初始化为 0)
 - 结束条件：14e5: cmp \$0x3, %rax (循环 3 次)
 - **推论：** 矩阵乘法的公共维度为 **3**。

矩阵A为2×3，矩阵B为3×2，结果为2×2的矩阵，均按照行优先存储

计算后把计算结果放在了0x10(%rsp)这个地址（也就是%rsp+16）开始的空间中，连续存放四个int，所以我们需要查看从这个地址开始的四个整数，就是这道题的答案。

以下是伪代码：

```
void phase_2(char *input) {
    int user_input[4];    // 存储在栈上 %rsp
    int calculated[4];    // 存储在栈上 %rsp+16

    // 全局矩阵，地址通过 lea 加载
    // matA 是 2x3 矩阵
    int matA[2][3] = { 6140 中的数据 };
    // matB 是 3x2 矩阵
    int matB[3][2] = { 6120 中的数据 };

    // 1. 读取输入 (sscanf)
```

```

// 必须成功读取 4 个整数，否则爆炸
if (sscanf(input, "%d %d %d %d", &user_input[0], &user_input[1],
&user_input[2], &user_input[3]) != 4) {
    explode_bomb();
}

// 2. 矩阵乘法计算 (Calculating Standard Answer)
// 对应汇编中的三层循环结构
int idx = 0;
for (int i = 0; i < 2; i++) {           // 外层循环 (r11d)
    for (int j = 0; j < 2; j++) {       // 内层循环 (r8d)
        int sum = 0;
        for (int k = 0; k < 3; k++) {   // 计算点积 (eax)
            sum += matA[i][k] * matB[k][j];
        }
        calculated[idx] = sum;          // 存入 %rsp+16 区域
        idx++;
    }
}

// 3. 比对结果
// 对应 1502 处的循环
for (int i = 0; i < 4; i++) {
    if (user_input[i] != calculated[i]) {
        explode_bomb();
    }
}
}

```

phase_3

1 96

这个题的整体是一个switch程序，根据不同第一个的输入（0-7），在每一个分支中进行不同的运算，然后要让运算结果等于输入的第二个数，才能拆除，具体如下：

第一阶段：检查输入

程序先执行一个输入，通过以下两行，判断出需要输入两个数

```

1558: 48 8d 4c 24 04      lea    0x4(%rsp),%rcx
155d: 48 89 e2             mov     %rsp,%rdx

```

```

156c: 83 f8 01             cmp     $0x1,%eax
156f: 7e 1d                jle     158e <phase_3+0x4a>

```

如果sscanf的返回值小于1，就爆炸，说明有多于1个的输入

```

1571: 83 3c 24 07          cmp     $0x7, (%rsp)
1575: 0f 87 c0 00 00 00    ja      163b <phase_3+0xf7>

```

然后把第一个输入a与7比较，必须小于7

第二阶段：跳转到每个分支

```
157b: 8b 04 24      mov    (%rsp),%eax
157e: 48 8d 15 9b 1c 00 00    lea    0x1c9b(%rip),%rdx    # 3220
<_IO_stdin_used+0x220>
1585: 48 63 04 82      movslq (%rdx,%rax,4),%rax
1589: 48 01 d0          add    %rdx,%rax
158c: ff e0            jmp    *%rax
```

把第一个输入a放入%eax，然后取一个固定的地址3320，这是跳转表的开头，通过读取每个分支的偏移量再加上开头地址，*%rax中存储的就是每个分支的地址了，跳转到该地址

第三阶段：每个分支中进行不同的计算

在每个分支结构中，操作都很类似，比如：

case 0:

```
1595: 8b 15 75 4b 00 00    mov    0x4b75(%rip),%edx    # 6110
<delta.1>
159b: b8 01 01 00 00      mov    $0x101,%eax
15a0: 29 d0               sub    %edx,%eax
```

case 1:

```
15cc: 8b 15 3e 4b 00 00    mov    0x4b3e(%rip),%edx    # 6110
<delta.1>
15d2: b8 b3 03 00 00      mov    $0x3b3,%eax
15d7: 29 d0               sub    %edx,%eax
15d9: eb c7               jmp    15a2 <phase_3+0x5e>
```

case 2

.....

case 7

每个分支都是：读取6110处delta.1的值，然后用不同的值（%eax）减delta.1，然后进行比较，代码如下：

```
15a2: 8b 54 24 04      mov    0x4(%rsp),%edx
15a6: 85 d2            test   %edx,%edx
15a8: 78 04            js     15ae <phase_3+0x6a>
15aa: 39 c2            cmp    %eax,%edx
15ac: 74 05            je     15b3 <phase_3+0x6f>
15ae: e8 05 09 00 00    call  1eb8 <explode_bomb>
15b3: 48 8b 44 24 08    mov    0x8(%rsp),%rax
15b8: 64 48 2b 04 25 28 00    sub    %fs:0x28,%rax
15bf: 00 00
15c1: 0f 85 83 00 00 00    jne    164a <phase_3+0x106>
15c7: 48 83 c4 18      add    $0x18,%rsp
15cb: c3              ret
```

0x4(%rsp)，也就是输入的第二个数b，不能小于0，并且把b与刚才算出来的eax比较，如果相等就运行到函数结尾（还有一些检查栈是否溢出的验证）返回，如果不相等就爆炸。

得到答案:

根据上面的分析, 第一个数是0-7任何一个都可以, 这里以1为例。在运行完这行代码后:

```
15cc:  8b 15 3e 4b 00 00      mov     0x4b3e(%rip),%edx
```

查看edx中的值, 也就是delta.1的值, 发现是851;

在case1中, $eax=0x3b3$, 所以第二个输入b就要等于 $0x3d3-851=96$, 所以第二个输入为96, 这个值也可以通过读取 **将b与eax (计算出来的值) 比较前的** `eax`寄存器的值来获得。

以下是伪代码:

```
void phase_3(char *input) {
    int x, y;
    int result;
    int delta = 851; // 内存 0x6110 <delta.1> 的值

    // 1. 读取输入, 要求必须有两个数
    if (sscanf(input, "%d %d", &x, &y) < 2) {
        explode_bomb();
    }

    // 2. 边界检查: x 必须 <= 7
    if (x > 7) {
        explode_bomb();
    }

    // 3. switch 跳转表逻辑
    switch (x) {
        case 0:
            result = 0x101 - delta;
            break;
        case 1:
            result = 0x3b3 - delta; // 947 - 851 = 96
            break;
        case 2:
            result = 0x94 - delta;
            break;
        case 3:
            result = 0x156 - delta;
            break;
        case 4:
            result = 0x364 - delta;
            break;
        case 5:
            result = 0x1d7 - delta;
            break;
        case 6:
            result = 0xe4 - delta;
            break;
        case 7:
            result = 0xf6 - delta;
            break;
        default:
            explode_bomb();
    }
}
```

```

// 4. 比对 y 和计算结果
if (y != result) {
    explode_bomb();
}
}

```

phase_4

31 AC

func4_1

这个函数转化为c代码如下：

```

int func4_1(int n){
    if(n<=0){
        return 0;
    }
    if(n==1){
        return 1;
    }
    return 2*func_4(n-1)+1;
}

```

每次把上一个结果乘2加一，实际计算的就是2的n次方-1；

func4_2

通过分析，这个函数一共有6个参数：(int h, int x, char a, char b, char c, ptr存放结果的指针)

edi:h, esi:x, edx:a, ecx:b, r8:c, r9:ptr （以下a,b,c只是参数名字，不代表真的字符内容）

模拟一个二叉树的中序遍历，来决定使用这三个字符参数中的哪些以及顺序。

第一部分：递归出口

1683:	41 89 d4	mov	%edx,%r12d
1686:	41 89 cd	mov	%ecx,%r13d
1689:	4c 89 cd	mov	%r9,%rbp
168c:	83 ff 01	cmp	\$0x1,%edi
168f:	74 2a	je	16bb <func4_2+0x46>
跳转:			
16bb:	88 55 00	mov	%d1,0x0(%rbp)
16be:	88 4d 01	mov	%c1,0x1(%rbp)
16c1:	41 c6 41 02 00	movb	\$0x0,0x2(%r9)
16c6:	48 83 c4 08	add	\$0x8,%rsp
16ca:	5b	pop	%rbx
16cb:	5d	pop	%rbp
16cc:	41 5c	pop	%r12
16ce:	41 5d	pop	%r13
16d0:	41 5e	pop	%r14
16d2:	41 5f	pop	%r15
16d4:	c3	ret	

如果h=1, 跳转到16bb, 把字符参数a,b和'\0'写入存放结果的地址;

第二部分: 计算左子树大小

```
1691:  89 f3          mov    %esi,%ebx
1693:  45 89 c6        mov    %r8d,%r14d
1696:  44 8d 7f ff     lea    -0x1(%rdi),%r15d
169a:  44 89 ff        mov    %r15d,%edi
169d:  e8 ad ff ff ff  call   164f <func4_1>
```

把h=h-1作为参数传入func4_1,返回值为val=2^(h-1) - 1;

第三部分: 比较与分支

分支A:

```
16a2:  39 d8          cmp    %ebx,%eax
16a4:  7d 2f          jge    16d5 <func4_2+0x60>
跳转:
16d5:  41 0f be ce    movsbl %r14b,%ecx
16d9:  41 0f be d4    movsbl %r12b,%edx
16dd:  49 89 e9       mov    %rbp,%r9
16e0:  45 0f be c5    movsbl %r13b,%r8d
16e4:  89 de         mov    %ebx,%esi
16e6:  44 89 ff       mov    %r15d,%edi
16e9:  e8 87 ff ff ff  call   1675 <func4_2>
```

如果参数x<=val, 跳到16d5, 这时根据上面的参数传递, %r14中存储的是字符参数c, %r12中存储的是字符参数a, %rbp始终是地址, %r13是字符参数b, 然后把这几个字符参数交换顺序, 从abc变成acb;

然后递归调用, 改变的是h (h和之前相比减1), 以及改变了三个字符的顺序。

分支B:

```
16a6:  8d 50 01       lea    0x1(%rax),%edx
16a9:  39 da          cmp    %ebx,%edx
16ab:  75 43          jne    16f0 <func4_2+0x7b>
跳转:
16f0:  41 0f be cd    movsbl %r13b,%ecx
16f4:  41 0f be d6    movsbl %r14b,%edx
16f8:  29 c3         sub    %eax,%ebx
16fa:  8d 73 ff       lea    -0x1(%rbx),%esi
16fd:  49 89 e9       mov    %rbp,%r9
1700:  45 0f be c4    movsbl %r12b,%r8d
1704:  44 89 ff       mov    %r15d,%edi
1707:  e8 69 ff ff ff  call   1675 <func4_2>
```

%edx=val+1 (2^(h-1)); 如果 x!=val+1, 就跳转至16f0;

字符轮换: 从abc变成cba; x=x-val-1; h=h-1; 然后递归调用

分支c:

如果上面两个分支都没有进入, 就运行到这里:


```

16ad: 44 88 65 00      mov     %r12b,0x0(%rbp)
16b1: 44 88 6d 01      mov     %r13b,0x1(%rbp)
16b5: c6 45 02 00      movb    $0x0,0x2(%rbp)
16b9: eb 0b           jmp     16c6 <func4_2+0x51>
跳转:
16c6: 48 83 c4 08      add     $0x8,%rsp
16ca: 5b             pop     %rbx
16cb: 5d             pop     %rbp
16cc: 41 5c           pop     %r12
16ce: 41 5d           pop     %r13
16d0: 41 5e           pop     %r14
16d2: 41 5f           pop     %r15
16d4: c3             ret

```

说明 $x=val+1$ ；把a、b和\0写入存结果的地址,函数结束；

总结：func4-2模拟了一个高度为 n 的字符变换树

输入：

- 树的高度 h。
- 目标索引 x。
- 三个字符池 a, b, c。
- 字符串缓冲区 str。

算法流程：

1. 计算左子树的大小：
 $val = 2^{\{h-1\}} - 1$
2. 判断目标 (x) 的位置：
 - 如果 (x 小于等于S)：目标在左子树。
 - 递归搜索左子树。
 - 字符集变为 (a,c,b)。
 - 如果 (x = val + 1)：目标就是当前节点。
 - 找到答案！将字符a 和b写入str。
 - 如果 (x > val + 1)：目标在右子树。
 - 更新索引 ($x' = x - (S + 1)$)。
 - 递归搜索右子树。
 - 字符集变为 (c,b,a)。

伪代码如下：

```

void func4_2(int h, int x, char a, char b, char c, char *ptr) {
    if (h==1) { //递归出口
        ptr[0]=c1;
        ptr[1]=c2;
        ptr[2]='\0';
        return;
    }

    int val=func4_1(h-1); // 左子树大小

    if (x<=val) {
        // 递归，交换b和c
        func4_2(h-1,x,a,c,b,ptr);
    }
}

```

```

}else if (x==val+1) {
    // 命中当前层，写入a和b
    ptr[0] = c1;
    ptr[1] = c2;
    ptr[2] = '\0';
}else {
    // 递归，减去偏移量，交换a和c
    func4_2(h-1, x-val-1,c3,c2,c1,ptr);
}
}

```

phase_4

```

1723:  48 8d 4c 24 10      lea    0x10(%rsp),%rcx
1728:  48 8d 54 24 0c      lea    0xc(%rsp),%rdx
172d:  48 8d 35 c9 1a 00 00 lea    0x1ac9(%rip),%rsi      # 31fd
<_IO_stdin_used+0x1fd>
1734:  e8 17 fa ff ff      call   1150 <__isoc99_sscanf@plt>
1739:  83 f8 02            cmp     $0x2,%eax
173c:  75 6d              jne     17ab <phase_4+0x9d>

```

输入两个数，存在 (%rdx) 和 (%rcx) ，检查输入个数，不是两个就爆炸

```

173e:  bf 05 00 00 00      mov     $0x5,%edi
1743:  e8 07 ff ff ff      call   164f <func4_1>
1748:  39 44 24 0c          cmp     %eax,0xc(%rsp)
174c:  75 64              jne     17b2 <phase_4+0xa4>

```

把5作为参数传入func4_1，返回2的5次方-1（31），如果第一个输入不等于31就爆炸，所以第一个输入的正确答案就是31；

```

174e:  48 8d 7c 24 10      lea     0x10(%rsp),%rdi
1753:  e8 de 04 00 00      call   1c36 <string_length>
1758:  83 f8 02            cmp     $0x2,%eax
175b:  75 5c              jne     17b9 <phase_4+0xab>

```

如果第二个输入的长度不等于2byte就爆炸；

```

175d:  48 8d 5c 24 14      lea     0x14(%rsp),%rbx
1762:  49 89 d9            mov     %rbx,%r9
1765:  41 b8 42 00 00 00    mov     $0x42,%r8d
176b:  b9 43 00 00 00      mov     $0x43,%ecx
1770:  ba 41 00 00 00      mov     $0x41,%edx
1775:  be 01 00 00 00      mov     $0x1,%esi
177a:  bf 05 00 00 00      mov     $0x5,%edi
177f:  e8 f1 fe ff ff      call   1675 <func4_2>

```

传参并调用func4_2，0x14(%rsp)也就是rsp+20作为func4_2写字符串的起始地址，也就是func4_2的第六个参数；剩下的前5个参数依次为：5，1，65（A），67（C），66（B）；

在func4_2中按照函数规则交换三个字符，最后把其中两个按一定顺序写入栈中

```

1784:  48 8d 7c 24 10      lea    0x10(%rsp),%rdi
1789:  48 89 de            mov    %rbx,%rsi
178c:  e8 c2 04 00 00      call   1c53 <strings_not_equal>
1791:  85 c0              test   %eax,%eax
1793:  75 2b              jne    17c0 <phase_4+0xb2>

```

把输入和func4_2的两个字符对比，如果不一样就爆炸。

后面的汇编就是一些对栈溢出的检测，和恢复寄存器值这种常规操作。

关于如何得到第二个正确答案，有以下两种方法：

- 在“178c: e8 c2 04 00 00 call 1c53 <strings_not_equal>”这里打断点，查看寄存器rsi中的值，就是“AC”
- 5, 1, 65 (A) , 67 (C) , 66 (B) 按照这5个输入，看func4_2的运行结果：
 - func4_2(5, 1, 'A', 'C', 'B'): ACB变成ABC;
 - func4_2(4, 1, 'A', 'B', 'C'): ABC变成ACB;
 - func4_2(3, 1, 'A', 'C', 'B'): ACB变成ABC;
 - func4_2(2, 1, 'A', 'B', 'C'): ABC变成ACB;
 - func4_2(1, 1, 'A', 'C', 'B'): 递归出口：结果字符串：“AC”

所以第二个正确输入是“AC”

伪代码如下：

```

void phase_4(char *input) {
    int num;
    char str[100];
    char expected_str[10];

    // 1. 读取输入
    if(sscanf(input, "%d %s", &num, str) != 2) explode_bomb();

    // 2. 验证整数
    // func4_1(5)=2^5-1=31
    if(num != func4_1(5)) explode_bomb();

    // 3. 验证字符串长度
    if(strlen(str) != 2) explode_bomb();

    // 4. 生成轮换字符串
    // 参数:h=5,x=1,chars='A','C','B'
    func4_2(5, 1, 'A', 'C', 'B', expected_str);

    // 5. 比对字符串
    if(strings_not_equal(str, expected_str)){
        explode_bomb();
    }
}

```

phase_5

dddddg

第一阶段：检查输入

```
17cc: 53                push    %rbx
17cd: 48 89 fb          mov     %rdi,%rbx
17d0: e8 61 04 00 00    call   1c36 <string_length>
17d5: 83 f8 06          cmp     $0x6,%eax
17d8: 75 2c            jne     1806 <phase_5+0x3a>
```

phase5只有一个参数在%rdi中，存储一个字符串地址ptr，这个字符串长度为6，否则爆炸；

第二阶段：准备

```
17da: 48 89 d8          mov     %rbx,%rax
17dd: 48 8d 7b 06       lea     0x6(%rbx),%rdi
17e1: b9 00 00 00 00    mov     $0x0,%ecx
17e6: 48 8d 35 53 1a 00 00 lea     0x1a53(%rip),%rsi    # 3240
<array.0>
```

%rax: ptr; %rdi: ptr+6; ecx作为累加器，先置0；把一个数组首地址放在%rsi中

第三阶段：循环

```
17ed: 0f b6 10          movzbl  (%rax),%edx
17f0: 83 e2 0f          and     $0xf,%edx
17f3: 03 0c 96          add     (%rsi,%rdx,4),%ecx
17f6: 48 83 c0 01       add     $0x1,%rax
17fa: 48 39 f8          cmp     %rdi,%rax
17fd: 75 ee            jne     17ed <phase_5+0x21>
17ff: 83 f9 3f          cmp     $0x3f,%ecx
1802: 75 09            jne     180d <phase_5+0x41>
1804: 5b              pop     %rbx
1805: c3              ret
```

取出字符串中的第一个字符放在%edx中，零扩展，并取低四bit；

把 (array.0地址+4*%rdx) 这个内存地址的值累加在ecx中，rax=rax+1，如果rax的值不等于0x6(%rbx)，就循环，循环结束后比较\$0x3f和%ecx，不相等就爆炸。

所以这个函数的功能就是，遍历这 6 个字符，取每个字符 ASCII 码的**低 4 位**作为索引，在整数数组 array.0中查找对应的数值，并将这些数值累加，累加的结果必须等于63。

所以我们查看array.0数组中的16个整数（因为只有4位索引，索引0-15），然后挑出六个数，使他们相加等于63，有很多答案，我选择了12×5+3，array.0数组内容如下：2，10，6，1，12，16，9，3，4，7，14，5，11，8，15，13。

12的索引是4，3的索引是7，所以要找到两个字符，使他们的 ASCII 码的16进制低4位分别为4和7，'d'的ASCII是0x64，'g'的ASCII是0x67，所以这个题的答案可以是dddddg

以下是伪代码：

```
void phase_5(char *input) {
    // 1.长度检查
```

```

if (strlen(input)!= 6) {
    explode_bomb();
}

int sum=0;
// array.0
int array_0[16]={2, 10, 6, 1, 12, 16, 9, 3, 4, 7, 14, 5, 11, 8, 15, 13};

// 2.循环累加
for (int i=0; i<6; i++) {
    // 取字符ASCII码的低4位作为索引
    int index=input[i]&0xF;
    // 查表并累加
    sum+=array_0[index];
}

// 3.结果检查
// 总和必须为63(0x3f)
if (sum!=63) {
    explode_bomb();
}
}

```

phase_6

3 6 4 2 1 5

第一阶段：检查输入

```

0000000000001814 <phase_6>:
.....
182e: 31 c0          xor     %eax,%eax
1830: 49 89 e5       mov     %rsp,%r13
1833: 4c 89 ee       mov     %r13,%rsi      #把%rsp作为第二个参数传入
1836: e8 3d 07 00 00 call    1f78 <read_six_numbers>
183b: 41 be 01 00 00 00 mov     $0x1,%r14d
1841: 49 89 e4       mov     %rsp,%r12
1844: eb 28         jmp     186e <phase_6+0x5a>

```

从输入读取六个数，依次放在栈顶；

```

184d: 48 83 c3 01    add     $0x1,%rbx
1851: 83 fb 05       cmp     $0x5,%ebx
1854: 7f 10         jg      1866 <phase_6+0x52>
1856: 41 8b 04 9c    mov     (%r12,%rbx,4),%eax
185a: 39 45 00       cmp     %eax,0x0(%rbp)  #把每个数和后面的数比较，
                        必须不同
185d: 75 ee         jne     184d <phase_6+0x39>
185f: e8 54 06 00 00 call    1eb8 <explode_bomb>
1864: eb e7         jmp     184d <phase_6+0x39>
1866: 49 83 c6 01    add     $0x1,%r14
186a: 49 83 c5 04    add     $0x4,%r13
186e: 4c 89 ed       mov     %r13,%rbp      #第一次从这里进入
1871: 41 8b 45 00    mov     0x0(%r13),%eax  #把输入的数字放入%eax
1875: 83 e8 01       sub     $0x1,%eax

```

1878:	83 f8 05	cmp	\$0x5,%eax	#数字减1后不能大于5, 所以数字要小于等于6
187b:	77 c9	ja	1846 <phase_6+0x32>	
187d:	41 83 fe 05	cmp	\$0x5,%r14d	
1881:	7f 05	jg	1888 <phase_6+0x74>	
1883:	4c 89 f3	mov	%r14,%rbx	
1886:	eb ce	jmp	1856 <phase_6+0x42>	

通过双重循环检查: %rbx控制内层循环 (指向正在判读的数的后面的数, %%rbp控制外层循环 (指向要判断的数))

1. 每个数都在 1-6 之间。(不能是负数的原因是ja是无符号数比较)
 2. 6 个数互不相同。
- 所以输入是 1-6 的全排列。

第二阶段: 取链表结点

1888:	be 00 00 00 00	mov	\$0x0,%esi	
188d:	8b 0c b4	mov	(%rsp,%rsi,4),%ecx	#循环: 把输入的数放入%ecx
1890:	b8 01 00 00 00	mov	\$0x1,%eax	
1895:	48 8d 15 84 49 00 00	lea	0x4984(%rip),%rdx	# 6220
<node1>				
189c:	83 f9 01	cmp	\$0x1,%ecx	
189f:	7e 0b	jle	18ac <phase_6+0x98>	#如果小于等于1, 取第一个结点地址
18a1:	48 8b 52 08	mov	0x8(%rdx),%rdx	#取下一个链表结点的地址
18a5:	83 c0 01	add	\$0x1,%eax	
18a8:	39 c8	cmp	%ecx,%eax	#比较结点的顺序号是否等于当前输入的数
18aa:	75 f5	jne	18a1 <phase_6+0x8d>	#不等于就走到下一个结点
18ac:	48 89 54 f4 20	mov	%rdx,0x20(%rsp,%rsi,8)	#等于就把结点地址写入%rsp+32开始的栈上的数组
18b1:	48 83 c6 01	add	\$0x1,%rsi	#循环控制器+1
18b5:	48 83 fe 06	cmp	\$0x6,%rsi	#循环6次
18b9:	75 d2	jne	188d <phase_6+0x79>	

第三阶段: 链表重连

18bb:	48 8b 5c 24 20	mov	0x20(%rsp),%rbx
18c0:	48 8b 44 24 28	mov	0x28(%rsp),%rax
18c5:	48 89 43 08	mov	%rax,0x8(%rbx)
18c9:	48 8b 54 24 30	mov	0x30(%rsp),%rdx
18ce:	48 89 50 08	mov	%rdx,0x8(%rax)
18d2:	48 8b 44 24 38	mov	0x38(%rsp),%rax
18d7:	48 89 42 08	mov	%rax,0x8(%rdx)
18db:	48 8b 54 24 40	mov	0x40(%rsp),%rdx
18e0:	48 89 50 08	mov	%rdx,0x8(%rax)
18e4:	48 8b 44 24 48	mov	0x48(%rsp),%rax
18e9:	48 89 42 08	mov	%rax,0x8(%rdx)
18ed:	48 c7 40 08 00 00 00	movq	\$0x0,0x8(%rax)

从%rsp+32开始, 按现在的栈上结点地址存放的顺序重新连接结点;

以前三行为例:

把第一个结点地址放在rbx，第二个放在rax，每个结点16个字节，后8字节是指针，指向下一个结点的首地址，所以在第一个结点地址加8的内存中放入rax（第二个结点的地址），后面的逻辑相同，在最后一个结点的next设置成了NULL。

第四阶段：排序检查

```
18f5:  bd 05 00 00 00      mov     $0x5,%ebp    #控制循环：比较5次
18fa:  eb 09               jmp     1905 <phase_6+0xf1>
18fc:  48 8b 5b 08         mov     0x8(%rbx),%rbx
1900:  83 ed 01            sub     $0x1,%ebp
1903:  74 11               je      1916 <phase_6+0x102>
1905:  48 8b 43 08         mov     0x8(%rbx),%rax
1909:  8b 00               mov     (%rax),%eax    #eax: 后一个结点的值
190b:  39 03               cmp     %eax, (%rbx)    #前一个结点的值必须小于后一个结点的值
190d:  7e ed               jle     18fc <phase_6+0xe8>
190f:  e8 a4 05 00 00      call    1eb8 <explode_bomb>
1914:  eb e6               jmp     18fc <phase_6+0xe8>
```

这一阶段按照刚才排好的顺序检查结点中值地大小，必须要升序排列，否则爆炸。

得到答案：

所以，输入的6个数决定了结点的顺序，我们需要查看结点中的值，把他们升序排列，答案就是它们对应的原始索引。

猜测结点在物理地址上不会离太远，所以先查看第一个结点# 6220 开始的24个int，发现前五个结点在物理上挨着，但是第六个结点不是：

```
(gdb) x/24d 0x55555555a220
0x55555555a220: 908   1   1431675440   21845
0x55555555a230: 568   2   1431675456   21845
0x55555555a240: 113   3   1431675472   21845
0x55555555a250: 483   4   1431675488   21845
0x55555555a260: 908   5   1431675248   21845
0x55555555a270: 0     0     0     0
```

然后根据第五个结点的next（1431675488 21845，也就是**0x55555555a170**）查看第六个结点的值：

```
0x55555555a170: 431   6     0     0
```

把他们按照值升序排列：113，431，483，568，908，908，对应他们的原始索引，得到答案：3 6 4 2 1 5

伪代码如下：

```
/* 根据汇编分析：value在偏移0，Next指针在偏移8 */
struct Node{
    int value;           // 0x0
    int unused_padding;  // 0x4
    struct Node *next;   // 0x8
};

extern struct Node node1;

void phase_6(char *input) {
    unsigned int indices[6];           // 输入的6个数字(栈上 %rsp)
```

```

struct Node *node_ptrs[6]; // 存储找到的6个节点指针(栈上 %rsp+0x20)

//1. 读取输入
read_six_numbers(input, indices);

//2. 输入检查: 必须是1-6的全排列
for(int i=0; i<6; i++){
    //必须在1到6之间
    if(indices[i]-1>5){
        explode_bomb();
    }
    //不能有相同的数字
    for(int j=i+1; j<6; j++){
        if(indices[i]==indices[j]){
            explode_bomb();
        }
    }
}

//3. 根据输入的索引找到对应的节点指针
for(int i=0; i<6; i++){
    int idx=indices[i];
    struct Node *temp=&node1;

    for(int k=1; k<idx; k++){
        temp=temp->next;
    }
    node_ptrs[i]=temp;
}

//4. 重链表: 根据新顺序修改next指针
struct Node *new_head=node_ptrs[0];

node_ptrs[0]->next=node_ptrs[1];
node_ptrs[1]->next=node_ptrs[2];
node_ptrs[2]->next=node_ptrs[3];
node_ptrs[3]->next=node_ptrs[4];
node_ptrs[4]->next=node_ptrs[5];
node_ptrs[5]->next=NULL;

//5. 排序检查: 遍历新链表, 确保按升序排列
struct Node *current = new_head;
for(int i=0; i<5; i++){
    //取出下一个节点的值进行比较
    if(current->value>current->next->value){
        explode_bomb(); // 如果当前值>下一个值, 爆炸
    }
    current=current->next;
}
}

```

secret_phase

cccaa

移动方向定义：

```
1957: c7 04 24 fe ff ff ff    movl    $0xfffffffffe, (%rsp)
195e: c7 44 24 04 ff ff ff    movl    $0xfffffffff, 0x4(%rsp)
.....
1a46: c7 44 24 78 ff ff ff    movl    $0xfffffffff, 0x78(%rsp)
1a4d: ff
1a4e: c7 44 24 7c 00 00 00    movl    $0x0, 0x7c(%rsp)
```

定义了：

DX 数组：占据了栈的 0 到 31 字节（共 8 个整数，每个 4 字节）。

DY 数组：占据了栈的 32 到 63 字节（即 0x20 开始）。

用不同索引来对应x, y的移动方向。

- Index 0: (-2, +1)
- Index 1: (-1, +2)
- Index 2: (+1, +2)
- Index 3: (+2, +1)
- Index 4: (+2, -1)
- Index 5: (+1, -2)
- Index 6: (-1, -2)
- Index 7: (-2, -1)

第一阶段：递归出口

```
1a56: 83 fe 04                cmp     $0x4,%esi
1a59: 75 6b                  jne     1ac6 <func7+0x18e>
1a5b: 83 fa 07                cmp     $0x7,%edx
1a5e: 75 66                  jne     1ac6 <func7+0x18e>
```

如果 x==4 且 y==7，说明走到了终点。

```
1a60: 49 63 c9                movslq  %r9d,%rcx
1a63: 0f b6 34 0f             movzbl  (%rdi,%rcx,1),%esi
1a67: b9 01 00 00 00          mov     $0x1,%ecx
1a6c: 40 84 f6                test    %sil,%sil    #检查当前字符是否是'\0'，字符串结束符
1a6f: 74 34                  je      1aa5 <func7+0x16d> #如果是'\0'，跳转到返回1
```

检查字符串是否结束；(如果没结束，后面逻辑会返回 0，意味着虽然到了终点但字符串没输完，视为失败)

第二阶段：步数检查与移动

检查：

```

1ac6:  b9 00 00 00 00      mov     $0x0,%ecx    #预设返回值为0
1acb:  41 83 f9 13          cmp     $0x13,%r9d
1acf:  7f d4                jg      1aa5 <func7+0x16d> #如果步数超过19, 返回
失败
1ad1:  49 63 c9             movslq  %r9d,%rcx
1ad4:  0f b6 34 0f          movzbl  (%rdi,%rcx,1),%esi
1ad8:  b9 00 00 00 00      mov     $0x0,%ecx
1add:  40 84 f6             test    %sil,%sil
1ae0:  74 c3                je      1aa5 <func7+0x16d> #字符串读完了还没到终
点, 返回失败
1ae2:  eb 98                jmp     1a7c <func7+0x144>

```

移动:

```

1a7c:  41 89 f2             mov     %esi,%r10d
1a7f:  41 83 e2 07          and     $0x7,%r10d
1a83:  83 e6 07             and     $0x7,%esi    #取低3位
1a86:  41 89 c0             mov     %eax,%r8d
1a89:  44 03 04 b4          add     (%rsp,%rsi,4),%r8d #移动x
1a8d:  41 89 d3             mov     %edx,%r11d
1a90:  44 03 5c b4 20       add     0x20(%rsp,%rsi,4),%r11d #移动y

```

第三阶段: 边界检查与障碍物检查

边界:

```

1a95:  44 89 c6             mov     %r8d,%esi
1a98:  44 09 de             or      %r11d,%esi
1a9b:  b9 00 00 00 00      mov     $0x0,%ecx
1aa0:  83 fe 07             cmp     $0x7,%esi    #检查(X|Y)是否>7
1aa3:  76 3f                jbe     1ae4 <func7+0x1ac>

```

如果xy均小于等于7, 或的结果也只在低3位

障碍物:

```

1b15:  48 8d 15 94 46 00 00  lea     0x4694(%rip),%rdx    # 61b0
<row0>
1b1c:  45 85 c0             test    %r8d,%r8d
1b1f:  7e 11                jle     1b32 <func7+0x1fa> #检查X是否<=0, 小于
零直接检查y
1b21:  b8 00 00 00 00      mov     $0x0,%eax    #循环计数器
1b26:  48 8b 52 08          mov     0x8(%rdx),%rdx    #%rdx=%rdx->next
1b2a:  83 c0 01             add     $0x1,%eax
1b2d:  41 39 c0             cmp     %eax,%r8d
1b30:  75 f4                jne     1b26 <func7+0x1ee> #循环结束: %rdx现在指
向第x行的地图
1b32:  49 63 c3             movslq  %r11d,%rax
1b35:  b9 00 00 00 00      mov     $0x0,%ecx
1b3a:  80 3c 02 01          cmpb    $0x1,(%rdx,%rax,1)    检查(%rdx+y)的字节
是不是1
1b3e:  0f 84 61 ff ff ff    je      1aa5 <func7+0x16d>

```

地图是一个链表数组, 先找到一行, 再通过偏移找列;

第四阶段：递归

```
1b44: 41 8d 49 01      lea    0x1(%r9),%ecx    #步数+1
1b48: 44 89 da          mov    %r11d,%edx      #Y坐标
1b4b: 44 89 c6          mov    %r8d,%esi       #X坐标
1b4e: e8 e5 fd ff ff    call   1938 <func7>     #递归调用
```

得到答案：

通过查看row0-row7得到棋盘：

行\列	0	1	2	3	4	5	6	7
0	0	0	1	0	0	1	0	0
1	0	0	0	1	0	0	0	1
2	1	0	1	0	0	1	0	0
3	1	0	0	0	0	0	0	0
4	0	1	0	0	1	0	1	0
5	1	0	0	1	1	0	0	0
6	0	0	0	0	0	1	0	1
7	0	1	0	0	0	0	0	0

其中一条路线是：(0, 0)——(2, 1)——(4, 2)——(6, 3)——(5, 5)——(4, 7)，然后去找每一步对应的索引。

伪代码：

```
// 内存中 row0 到 row7 的结构
struct Node {
    char map[8];          // 偏移 0-7: 8个字节的地图数据 (1=障碍, 0=通路)
    struct Node *next;    // 偏移 8-15: 指向下一行节点的指针
};

// 全局变量
struct Node *row0_head;

// 参数映射:
// input_str: %rdi (输入字符串)
// x:         %esi (当前 X 坐标)
// y:         %edx (当前 Y 坐标)
// step:      %ecx (当前步数/字符串索引)
int func7(char *input_str, int x, int y, int step){

    // 1.栈上的位移表
    int dx[8]={-2,-1, 1, 2, 2, 1,-1,-2};
    int dy[8]={ 1, 2, 2, 1,-1,-2,-2,-1};

    // 2.检查是否到达终点
    if(x==4&&y==7){
        // 如果到了终点, 且字符串也刚好读完 ('\0'), 则成功
        if(input_str[step]=='\0'){
            return 1; // 返回 1
        }
    }
}
```

```

    }
    return 0; // 到了终点但字符串没输完，视为失败
}

// 3.失败条件检查
if(step>19){
    return 0; // 步数超过限制
}

if(input_str[step]=='\0'){
    return 0; // 字符串读完了，但还没到终点(x!=4 || y!=7)
}

// 4.计算下一步移动
int char_code=(int)input_str[step];
int move_index=char_code&0x7; // 取字符 ASCII 码最后 3 位

// 查表得到新坐标
int next_x=x+dx[move_index];
int next_y=y+dy[move_index];

// 5.边界检查
if(next_x<0||next_x>7||next_y<0||next_y>7){
    return 0; // 越界，爆炸
}

// 6.障碍物碰撞检查
struct Node *ptr = row0_head; // 获取链表头(row0)

// 沿着next指针跳跃next_x次，找到对应的那一行
for(int i=0;i<next_x;i++) {
    ptr=ptr->next;
}

// 检查ptr->map[next_y]是否为1
if(ptr->map[next_y]==1){
    return 0; // 撞墙，爆炸
}

return func7(input_str,next_x,next_y,step+1);
}

void secret_phase() {
    char *input;

    // 读取输入
    input=read_line();

    // 检查长度
    int len=string_length(input);
    if(len>20){
        explode_bomb();
    }

    // 调用递归搜索，初始状态：x=0, y=0, step=0
    int result = func7(input, 0, 0, 0);

    // 检查结果

```

```
if(result==0){
    explode_bomb(); // 返回 0 则爆炸
}else {
    phase_defused();
}
}
```

反馈/收获/感悟/总结

做了这个lab以后对汇编语句的熟悉有了质的飞跃。

参考的重要资料

教材CS:APP